

AI-Assisted Development Reflection

PRISM: Agentic RAG for Codebase Intelligence

1. Introduction

PRISM is an Agentic RAG system for codebase intelligence. It indexes a repository from a read-only GitHub App installation or a ZIP upload, parses Python, JavaScript, TypeScript, JSX and TSX with Tree-sitter, retrieves evidence with combined semantic, lexical and symbol search, and answers questions, traces flows across files and assesses change impact. Every citation is validated against the evidence the model was actually shown. The finished system has a FastAPI backend, a React and Vite frontend, 18 PostgreSQL tables, eleven registered agent tools and a single bounded controller.

AI assistance ran through the whole project. The specification was drafted and refined in conversation with an AI assistant, implementation was delegated to Codex across 30 logged phases, and the same tool helped write the tests, the evaluation harness, the security review, the deployment checks and the final documentation. This reflection describes what that collaboration looked like in practice: where it saved effort, where it produced work that was wrong or incomplete, and what I had to verify myself.

A note on evidence. `prompts.md` records each phase's prompt, a summary of the output and the test result. It does not contain debugging transcripts, and the fields for my own acceptance and modification decisions are manual placeholders, because Git history cannot reconstruct them. I therefore limit claims about failures to what the log and the repository show, and I mark inference as inference.

2. How AI Was Used During Development

I organised the work as 30 bounded phases, numbered 0 to 29. Almost every recorded prompt has the same shape: read named specification sections, inspect named files, implement a list, keep specific areas out of scope, and update the log and README status only after all tests pass. Within that structure the AI played five roles.

- **Specification drafting and revision.** The specification runs to more than 1,400 lines. After review feedback from my mentor it gained scope limits (section 2.6), per-feature success criteria (section 30.0), a privacy policy (section 28.3) and extra acceptance criteria. It was also revised during implementation, for example to describe how `/ask` routes external documentation questions (section 16.5).
- **Subsystem implementation.** ZIP and GitHub ingestion, parsing, chunking, embeddings, hybrid retrieval, evidence building, memory, plugins, the tool registry, output validation, the agent controller, and the Q&A, flow trace, change impact, architecture and comparison features (Phases 5 to 22).
- **Cross-layer work.** REST completion and the two frontend phases (Phases 23 to 25).
- **Verification tooling.** A demo repository with a deterministic evaluation, a security review with 17 test cases, and a stack smoke test (Phases 26 to 28).
- **Documentation and review.** The README rewrite, a 60-item acceptance checklist and a demo script (Phase 29).

The architectural decisions were fixed before code was written: one custom controller instead of a framework, two peer model slots with no automatic fallback, deterministic evaluation with no LLM judge. Acceptance stayed with me. The AI worked inside those limits.

3. What AI Did Well

Turning a frozen specification into code

The clearest success was hybrid retrieval in Phase 11. Section 14.1 of the specification froze the parameters: weights of 0.5 semantic, 0.3 lexical and 0.2 symbol presence, a 0.5 exact-symbol boost, and explicit deduplication and adjacent-chunk rules. The implementation reproduces them, and a test named for the frozen weights checks them. This worked because no design decision was left to make, and correctness could be checked against a table. By contrast, where the specification only said memory should be meaningfully reused, the acceptance checklist still marks it partial, because reuse across follow-up questions has not been demonstrated.

Repetitive engineering

Phases 0 to 3 produced the scaffold, authentication, Alembic setup, the source models and the provider abstraction. Phase 1 alone delivered password hashing, JWT handling, protected-route tests and a migration verified against a fresh database. On the frontend it produced a typed API client, query hooks and validated forms, and the suite reached 40 tests. These tasks suit AI because the patterns are common, the design content is low and tests verify the result quickly.

Cross-layer changes

In Phase 24 the GitHub repository picker needed branch selection. The log records that the AI added a narrow backend route for it, together with an authorization test, rather than working around the gap in the frontend. In Phase 25 it cleared cached repository data on login and sign-out, with a test, so results cannot carry across accounts; the recorded prompt does not mention this.

Tests and deterministic evaluation

Phase 26 built a FastAPI and React fixture repository and a runner that pushes 25 recorded questions through the real ingestion, retrieval and controller code. It uses hash embeddings and MockProvider, so it needs no network or credentials. Phase 27 turned the threat list in section 28.1 into tests that enumerate every registered tool and every repository route with a non-owner identity, and an AST check that rejects interpolated SQL. Exhaustive enumeration against a written threat list suits AI well.

Honest status reporting

Because the prompts said to update status only after tests pass, the AI held to that rule when it was inconvenient. Phase 26 left the README at its Phase 25 status when the sandbox blocked verification, and Phase 29 withheld the "MVP complete" line because live GitHub, model and web checks and a recorded demo were outstanding. The acceptance checklist lists those items as partial or manual.

4. Where AI Failed or Struggled

Not every problem in the log is an AI limitation. I separate the two below, because treating ordinary engineering friction as model failure would distort the lessons.

Defects that passing tests did not reveal

By the end of Phase 25 the backend suite had 278 passing tests. The Phase 26 benchmark then exposed two grounding defects. The deterministic flow-trace fallback produced citations that were incompatible with the citation validator, and evidence quality was wrongly classified as NONE when a queried exact symbol sat inside a merged chunk. The second defect would have made PRISM refuse a question with an insufficient-evidence error about a symbol it had indexed. My inference is that tests written by the same tool for the same code share its assumptions, while a benchmark built from ground truth recorded from the fixture does not. That is inference, not something the log states.

Security gaps in generated code

The Phase 27 review found two gaps. `search_web` did not check repository ownership before touching its cache or provider, and file reading did not apply the `secret-filename` denylist to an artificially seeded row. The second needed a row that ingestion would never create, so it was defence in depth. The first was a real missing check. The Phase 14 prompt asked for authenticated file tools and an "independently callable" search tool, so the gap may owe as much to my wording as to the model, though I cannot show that from the log. Either way, a per-feature test does not establish a cross-cutting property such as ownership. Only a review that enumerated every tool found it.

Sequencing mistakes and accumulated residue

In Phase 9 the composite and IVFFlat indexes had been created prematurely in revision 0007. They were moved to revision 0008, which conditionally removes the earlier indexes so that both existing and fresh databases upgrade. The phased build also left residue: the Phase 4 `memory_items` scaffold table was superseded in Phase 13 but remains in the migration history and in the 18-table inventory, and the supplementary-document tables in the specification were never built.

Drift that the log does not explain

The Phase 11 and 17 entries record a structural expansion cap of 15 chunks and bound tests at 8 tool iterations, 3 structural rounds and 2 web searches. The current specification, code and tests say 24, 12, 5 and 3. The log does not say when or why these changed. Either the code deviated from a frozen parameter and was later corrected, or the specification was revised and the log was not. I cannot tell which, and I only noticed while preparing this reflection. Phase 25 also depends on backend patches that have no entry of their own, and migrations 0013 and 0015 are not mentioned in the log. This is largely a process failure on my side, and it shows how easily frozen parameters can move unrecorded.

Scope discipline

Scope held well. Each entry from Phase 17 onward records what was left out, and the additions that were made had a stated reason, such as the `messages` table needed for session memory. If anything the build erred toward doing less: the FAILED indexing screen says retry is unavailable, and GitHub installation disconnect was never implemented. I cannot say whether the AI would have expanded scope without the explicit out-of-scope lines, because I never ran that experiment.

Problems that were ordinary engineering, not AI failure

The first Phase 28 smoke question returned 422 without a model call because it was too broad for the evidence gate. That was the gate working as specified. The fix was to ask about the exact `create_access_token` symbol, though it also shows a real product characteristic: a broad question with weak evidence can be refused. One model `ConnectError` passed on retry with no code change, and SQLAlchemy's reserved metadata name forced the attribute `chunk_metadata`. A sandbox permission error and an approval usage limit delayed Phase 26 verification, and tracked `node_modules` contents did not match the lockfile in Phases 20 and 21. None of these is a reasoning failure by the model.

5. The Importance of Verification

The last recorded runs show 357 backend tests passing with two opt-in PostgreSQL cases skipped, 40 frontend tests passing, a 26-question deterministic evaluation with zero invalid references, 17 security cases, and a stack smoke test that registered an account, imported a ZIP, waited for the index to become ready, asked a grounded question through a configured real model and checked the persisted trace. Core-module coverage was 92.52% when last measured in Phase 27.

Two events show the difference between code that looks correct and code that has been demonstrated to behave correctly. The Phase 26 benchmark found defects that 278 tests had not. The Phase 28 smoke test was the first time the whole path ran through a real model, and it is where the too-broad question surfaced. Reading the code would not have found either one.

What has not been demonstrated matters just as much. Every GitHub test uses a mocked transport, and no test contacted GitHub, so the live App flow is marked manual in the checklist. The two pgvector integration tests were skipped in every logged run, so PostgreSQL vector search is only partly verified. The benchmark uses hash embeddings and a mock model, so it measures the pipeline and not any production model. It also grew from 25 to 26 questions after its first run, which means its perfect scores on a 29-file fixture are a regression gate, not a quality claim. No recorded live demo exists yet.

This is the principle PRISM itself enforces. The system discards a citation that cannot be traced to evidence it was given, and I tried to hold my own work to the same rule. A phase counted as done when a test or measurement passed, and a status line was withheld when it had not. Plausibility is not evidence, for a model answer or for a generated module.

6. How My Prompting Changed

The recorded prompts shrank as the project progressed. Those for Phases 0 to 9 run from about 340 to 650 words, with numbered deliverables, explicit out-of-scope lists, acceptance conditions and, in Phase 0, a demand for the exact verification commands. The Phase 6 prompt constrained structure as well as behaviour: GitHub and ZIP imports had to use one normalization function with no branching on source type, and the finished code and acceptance checklist confirm that they share a pipeline. Phases 10 to 20 run about 100 to 200 words and lean on section numbers and named files. Phases 21 to 29 are mostly 35 to 80 words.

I read this as context moving out of the prompt and into the specification and the existing code. What did not change were the two gates, "out of scope" and "only after all tests pass". The later prompts also change character. Phases 26 to 29 ask for review, measurement and fixing discovered gaps, with clauses such as "no

new product feature is in scope" and "only with observed results". The shorter prompts had a cost. They pushed more responsibility onto my review of the output, and the log lost detail, which is why my acceptance decisions are placeholders. I also cannot confirm that the logged wording is the original in every case.

7. Lessons Learned

Specification quality sets the ceiling. Frozen parameters produced exact implementations. The sections added after review, the success criteria, privacy policy and scope limits, gave the evaluation and security phases something concrete to test against. Where the specification was soft, as with memory reuse, the outcome was partial.

Bounded tasks are reliable. Each of the 30 phases ended with a runnable suite, and the logged backend test count grew at every step, from 1 to 357, without a recorded regression in earlier suites.

Tests are the acceptance mechanism, but they need an independent source of truth. The AI wrote the code and most of its tests, and the ground-truth benchmark caught what those tests could not.

Cross-cutting properties need whole-system tests. Ownership, secret exclusion and repository isolation passed feature tests and still had gaps until a test enumerated every tool and route.

Naming files to inspect works. Prompts that named existing modules produced changes that fit the structure, such as one shared import path and one controller reused for flow tracing and impact analysis.

Judgement stayed with me. I decided to keep the 2,000-file target as a warning rather than a rejection, to use no LLM judge, to defer the optional code-review endpoint (the Phase 23 entry says this was by my direction), and to withhold the MVP-complete status. No prompt made those decisions.

8. What I Would Do Differently

- **Build the evaluation harness earlier.** The two grounding defects lived in code from Phases 12 and 19 for several days before Phase 26 found them. A small benchmark after the controller existed would have caught them sooner.
- **Test security properties from the day tools appear.** An ownership test over the registry in Phase 4 would have covered every later tool automatically.
- **Keep frozen parameters in one place.** A single constants module checked against the specification, and a logged entry for every change, would have prevented the drift described above.
- **Record my own decisions at the time.** Writing accepted and modified fields in the same session, and logging interstitial patches, would have made this reflection easier to support and the log more useful.
- **Exercise the external and optional paths earlier.** The pgvector tests should run routinely, and a live GitHub App check should come early, because a mocked transport can only reproduce what I already believed about GitHub.

9. Final Reflection

AI made a project of PRISM's breadth practical in under two weeks. It turned a detailed specification into working subsystems, wrote much of the test suite and built the tooling that measured its own output, but it did not build the project by itself. The quality of the result came from the combination: a specification that fixed the architecture and the success criteria, review that added evaluation and privacy requirements early, phases small enough to verify, deterministic tests, a security review that found gaps the feature tests missed, and a habit of refusing to call work finished without evidence. I remained responsible for scope, correctness, verification and acceptance, and the parts of the system I can least vouch for are the ones I could not verify, which the acceptance checklist lists openly.